

CSS Advanced

Week 3 · Friday Session

→ CSS Position

→ Pseudo-classes & Pseudo-elements

→ Transitions & Animations

→ Custom Properties

→ Media Queries

→ CSS Grid Introduction

Today's Agenda

WEEK 3 · FRIDAY

What we cover in this Friday 2-hour session

CSS Position

static · relative · absolute · fixed · sticky

CSS Custom Properties

--variables · var() · theming with :root

Pseudo-classes & Elements

:hover · :focus · :nth-child · ::before · ::after

Media Queries

@media · responsive breakpoints · mobile-first

Transitions & Animations

transition · @keyframes · animation shorthand

CSS Grid Introduction

display:grid · template-columns · gap

CSS Position

WEEK 3 · FRIDAY

Five values that control how an element is placed in the layout

position:
static

Default — normal flow
No
top/left/right/bottom
effect

position:
relative

Normal flow THEN
offset
from its own original
spot

position:
absolute

Removed from flow
Anchored to nearest
positioned parent

position:
fixed

Removed from flow
Stays fixed to the
VIEWPORT

position:
sticky

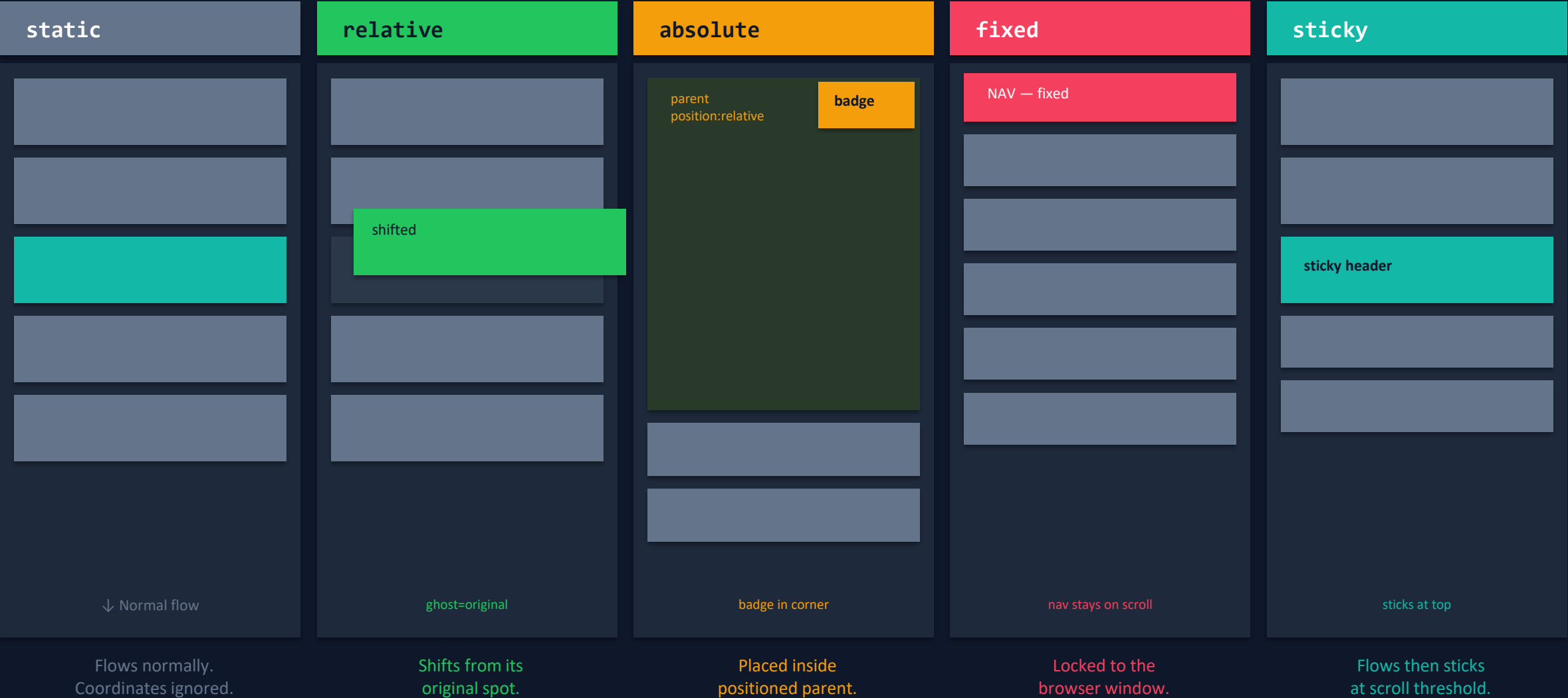
Relative until scroll
threshold → then fixed

CSS

```
/* position affects HOW coordinates apply */
.elem {
  position: absolute;
  top: 20px;
  right: 20px;
}
```

Position — Visual Diagram

See exactly where each value places an element



absolute vs fixed — The Key Difference

One anchors to a parent, the other to the browser window

position: absolute

- Removed from normal flow
- Positioned relative to nearest POSITIONED ancestor (parent with position ≠ static)
- If no positioned ancestor → positions relative to <body>
- SCROLLS with the page

CSS

```
.parent { position: relative; }  
.tooltip { position: absolute;  
  top: 0; right: 0; }
```

position: fixed

- Removed from normal flow
- Positioned relative to the VIEWPORT (browser window)
- Does NOT move when the page scrolls
- Perfect for sticky navbars, chat buttons, cookie banners

CSS

```
nav {  
  position: fixed;  
  top: 0; left: 0;  
  width: 100%;  
}
```

position: sticky — The Smart One

WEEK 3 · FRIDAY

Behaves as relative until a threshold → then acts like fixed

`position: sticky` is a HYBRID. Before the element reaches the specified offset during scrolling it sits in the normal flow (like relative). Once the scroll position hits the threshold (e.g., `top: 0`), it sticks in place (like fixed) — but ONLY within its parent container.

CSS

```
.section-title {  
  position: sticky;  
  top: 60px;    /* sticks 60px from top of viewport */  
  background: #0f766e;  
  z-index: 10;  
}
```

Common uses

- Table headers visible while scrolling rows
- Section labels in long scrollable lists
- Sidebar that sticks until section ends
- Progress bar at top after scroll

Sticky — Before & After Scroll

WEEK 3 · FRIDAY

Two states of the same element as you scroll down

BEFORE scrolling

Content block

Content block

STICKY HEADER (in flow)

Content below header

Content below header

Content below header

`position: sticky` (acts like relative)

scroll ↓

AFTER scroll threshold

STICKY HEADER (stuck at top:60px)

Content block 3

Content block 4

Content block 5

Content block 6

Content block 7

`position: sticky` (acts like fixed)

Check-In · CSS Position

WEEK 3 · FRIDAY

Q1–5 · Answer each before reading the next one

Q1 What is the DEFAULT value of the CSS position property?

- ☐ static
- ☐ relative
- ☐ absolute
- ☐ fixed

Q2 position: relative offsets the element relative to ...

- ☐ its nearest positioned ancestor
- ☐ the viewport
- ☐ its own original position
- ☐ the document <body>

Q3 position: absolute anchors the element relative to ...

- ☐ the viewport
- ☐ its nearest POSITIONED ancestor (or <body>)
- ☐ the document root
- ☐ the screen

Q4 Which position value stays fixed to the viewport while scrolling?

- ☐ absolute
- ☐ relative
- ☒ fixed
- ☐ sticky

Q5 position: sticky switches from relative to fixed when ...

- ☐ the element is clicked
- ☐ z-index becomes positive
- ☐ the scroll reaches the element's threshold
- ☐ JavaScript sets it

Check-In · CSS Position

WEEK 3 · FRIDAY

Q1–5 · Answer each before reading the next one

Q1 What is the DEFAULT value of the CSS position property?

✓ **static**

relative
absolute
fixed

Q2 position: relative offsets the element relative to ...

its nearest positioned ancestor
the viewport

✓ **its own original position**

the document <body>

Q3 position: absolute anchors the element relative to ...

the viewport

✓ **its nearest POSITIONED ancestor (or <body>)**

the document root
the screen

Q4 Which position value stays fixed to the viewport while scrolling?

absolute
relative

✓ **fixed**

sticky

Q5 position: sticky switches from relative to fixed when ...

the element is clicked
z-index becomes positive

✓ **the scroll reaches the element's threshold**

JavaScript sets it

CSS Pseudo-classes

WEEK 3 · FRIDAY

Style an element based on its STATE — no JavaScript needed

:hover

User hovers the mouse over the element

```
a:hover { color: teal; }
```

:focus

Element receives keyboard/click focus

```
input:focus { outline: 2px solid teal; }
```

:visited

Link has already been visited

```
a:visited { color: purple; }
```

:active

Element is being clicked RIGHT NOW

```
button:active { transform: scale(0.97); }
```

:first-child

First child of its parent

```
li:first-child { font-weight: bold; }
```

:nth-child(n)

Every nth child (2n=even, 2n+1=odd)

```
tr:nth-child(even) { background: #f0fdfa; }
```

Pseudo-classes — Button State Gallery

WEEK 3 · FRIDAY

One button, four CSS states — all styled with pure CSS

Normal

Trigger: (no pseudo-class)

Click Me

CSS

```
.btn {
  background: #0f766e;
  color: white;
  padding: 12px 24px;
  transition: all 0.25s;
}
```

:hover

Trigger: mouse over

Click Me

CSS

```
.btn:hover {
  background: #14b8a6;
  transform:
    translateY(-2px);
}
```

:focus

Trigger: Tab key / click

Click Me

CSS

```
.btn:focus {
  outline: 3px solid #F59E0B;
  outline-offset: 2px;
}
```

:active

Trigger: while clicking

Click Me

CSS

```
.btn:active {
  background: #095C56;
  transform:
    scale(0.97);
}
```

How to read this:

All four states are the SAME button — same HTML, same base CSS. The pseudo-class selector adds extra styles ONLY while that state is active. The transition on the base .btn makes the change animate smoothly. Remove transition and the change is instant — try both in the browser!

CSS Pseudo-elements

WEEK 3 · FRIDAY

::before & ::after — insert virtual elements with CSS alone

::before — content BEFORE the element's text

- Inserts a virtual element BEFORE the content
- Must have content: property to render
- Can be styled just like a real element
- No extra HTML needed

CSS

```
h2::before {  
  content: ' 🌿 ';  
  color: #14b8a6;  
}  
/* Renders: 🌿 My Heading */
```

::after — content AFTER the element's text

- Inserts a virtual element AFTER the content
- Same rules: must have content: property
- Popular for decorative lines, badges, icons
- Used in CSS-only tooltip patterns

CSS

```
.required::after {  
  content: ' *';  
  color: red;  
  font-size: 0.9em;  
}
```

::before & ::after — Where Content Goes

CSS inserts virtual elements around the real content

How the browser renders an element with ::before and ::after

::before

Created by CSS
content: ' 🌿 '



Element content

The real HTML text
'My Heading'



::after

Created by CSS
content: ' * '

Real examples

Example 1 — Icon before heading

HTML: `<h2>About Us</h2>`

CSS: `h2::before { content: ' 🌿 '; }`

Result: 🌿 About Us

No extra or icon tag needed in the HTML!

Change all icons site-wide by editing ONE CSS rule.

Example 2 — Required field asterisk

HTML: `<label class="req">Email</label>`

`.req::after { content: ' *'; color:red; }`

Result: Email *

*No * in the HTML — CSS adds it automatically.*

Remove class='req' and the asterisk disappears.

Check-In · Pseudo-classes, z-index, overflow

WEEK 3 · FRIDAY

Q6–12

Q6 z-index controls ...

stacking order (which element is on top)
element zoom
opacity
3D depth in perspective

Q7 overflow: hidden on a container ...

clips content beyond the container bounds
hides the element completely
makes content scrollable
collapses the height to zero

Q8 Which pseudo-class fires when a mouse hovers?

:hover
:focus
:active
:visited

Q9 :focus applies when ...

the element receives keyboard or click focus
the mouse hovers
a link was already visited
the element is the first child

Q10 :nth-child(2n) selects ...

every even-numbered child
every 2nd type element
children after the 2nd
only the 2nd child

Q11 ::before inserts content ...

before the element's own text/content
before the parent element
as the last child
as a sibling after the element

Q12 ::before and ::after MUST always have which property to be visible?

content: (even if empty string "")
display: block
position: absolute
z-index: 1

Check-In · Pseudo-classes, z-index, overflow

WEEK 3 · FRIDAY

Q6–12

Q6 z-index controls ...

✓ **stacking order (which element is on top)**

element zoom
opacity
3D depth in perspective

Q7 overflow: hidden on a container ...

✓ **clips content beyond the container bounds**

hides the element completely
makes content scrollable
collapses the height to zero

Q8 Which pseudo-class fires when a mouse hovers?

✓ **:hover**

:focus
:active
:visited

Q9 :focus applies when ...

✓ **the element receives keyboard or click focus**

the mouse hovers
a link was already visited
the element is the first child

Q10 :nth-child(2n) selects ...

✓ **every even-numbered child**

every 2nd type element
children after the 2nd
only the 2nd child

Q11 ::before inserts content ...

✓ **before the element's own text/content**

before the parent element
as the last child
as a sibling after the element

Q12 ::before and ::after MUST always have which property to be visible?

✓ **content: (even if empty string "")**

display: block
position: absolute
z-index: 1

CSS Transitions

WEEK 3 · FRIDAY

Smoothly animate a CSS property change over time

transition: property duration timing-function delay;

property

WHAT to animate
(background-color, transform,
opacity ...)

duration

HOW LONG (0.3s, 500ms ...)

timing

EASING CURVE
(ease · linear · ease-in · ease-out)

delay

WAIT before starting
(0s, 200ms ...)

CSS

```
.btn {  
  background: #0f766e;  
  transition: background 0.25s ease, transform 0.15s ease;  
}  
.btn:hover {  
  background: #14b8a6;  
  transform: translateY(-2px);  
}
```

Best practice

→ Use 0.2s–0.4s for UI transitions
(fast enough to feel instant, slow enough to see)

→ transition: all works but is slow
— name the specific property

→ Put transition on BASE state, not :hover
(so it animates in BOTH directions)

Transition — Right vs Wrong Placement

Where you write transition changes how it behaves

✗ WRONG — transition in :hover only

```
CSS
.btn {
  background: #0f766e;
  /* NO transition here */
}
.btn:hover {
  background: #14b8a6;
  transition: background 0.3s ease;
  /* WRONG PLACE! */
}
```

What happens:

→ Mouse ENTERS: smooth animation (0.3s)

→ Mouse LEAVES: INSTANT JUMP back! (broken)

Because :hover styles (including transition) are removed when mouse leaves

✓ CORRECT — transition on base element

```
CSS
.btn {
  background: #0f766e;
  transition: background 0.3s ease;
  /* CORRECT PLACE! */
}
.btn:hover {
  background: #14b8a6;
  /* only the changed value */
}
```

What happens:

→ Mouse ENTERS: smooth animation (0.3s)

→ Mouse LEAVES: smooth animation back (0.3s)

transition stays on .btn always, so both directions animate

CSS Animations — @keyframes

WEEK 3 · FRIDAY

Multi-step animations that run automatically, no JavaScript needed

transition = A → B (triggered by state change) animation = A → B → C → ... (runs on its own)

CSS

```
/* STEP 1: define the keyframe sequence */
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(20px); }
  to   { opacity: 1; transform: translateY(0); }
}

/* STEP 2: attach to the element */
.hero {
  animation: fadeIn 0.6s ease-out forwards;
}

/* Multiple stops */
@keyframes pulse {
  0%, 100% { transform: scale(1); }
  50%     { transform: scale(1.05); }
}
```

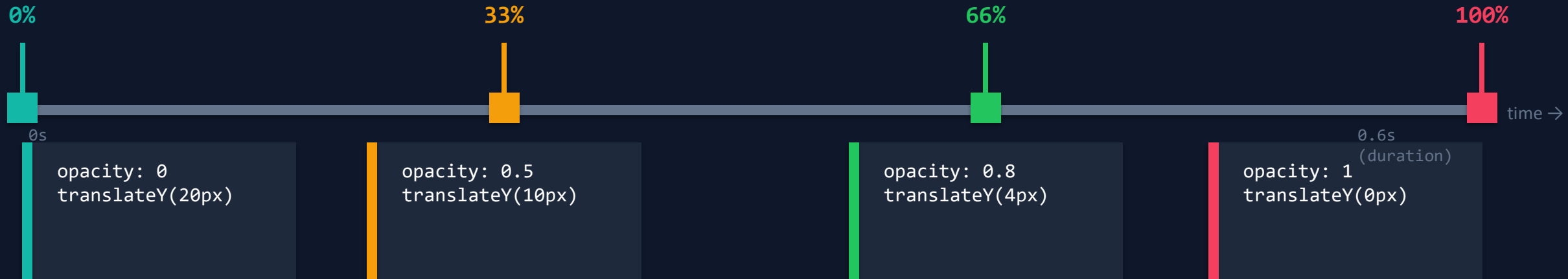
animation shorthand parts

name	fadeIn — must match @keyframes name
duration	0.6s — how long one cycle takes
timing	ease-out · linear · cubic-bezier(...)
delay	0s — wait before starting
iteration	1 · infinite · 3
direction	normal · reverse · alternate
fill-mode	forwards = keep final state none · backwards · both

Animation Timeline — @keyframes Visualized

WEEK 3 · FRIDAY

How the browser interprets keyframe stops



CSS

```
@keyframes fadeIn {  
  from { opacity: 0; transform: translateY(20px); }  
  33% { opacity: 0.5; transform: translateY(10px); }  
  66% { opacity: 0.8; transform: translateY(4px); }  
  to   { opacity: 1; transform: translateY(0); }  
}  
.hero { animation: fadeIn 0.6s ease-out forwards; }
```

fill-mode: forwards

WITHOUT forwards:

Element returns to opacity:0 after animation ends (invisible!)

WITH forwards:

Element keeps the FINAL keyframe state (opacity:1) after animation

Check-In · Transitions, Animations, Variables

WEEK 3 · FRIDAY

Q13–17

Q13 The CSS transition property animates changes in ...

- CSS property values when they change
- HTML element positions
- JavaScript variable values
- DOM structure

Q14 Which at-rule defines a CSS animation's keyframe sequence?

- @keyframes
- @media
- @font-face
- @animate

Q15 CSS custom properties (variables) are declared with the syntax ...

- name: value;
- \$name: value;
- var(name): value;
- name = value;

Q16 To USE a CSS variable called --primary, the syntax is ...

- var(--primary)
- \$(--primary)
- #primary
- %(primary)

Q17 @media (max-width: 768px) { } applies styles when ...

- the viewport is 768px wide or NARROWER
- the viewport is exactly 768px
- the font is 768px
- the device is mobile-only

Check-In · Transitions, Animations, Variables

WEEK 3 · FRIDAY

Q13–17

Q13 The CSS transition property animates changes in ...

✓ **CSS property values when they change**

- HTML element positions
- JavaScript variable values
- DOM structure

Q14 Which at-rule defines a CSS animation's keyframe sequence?

✓ **@keyframes**

- @media
- @font-face
- @animate

Q15 CSS custom properties (variables) are declared with the syntax ...

✓ **--name: value;**

- \$name: value;
- var(name): value;
- name = value;

Q16 To USE a CSS variable called --primary, the syntax is ...

✓ **var(--primary)**

- \$(--primary)
- #primary
- %(primary)

Q17 @media (max-width: 768px) { } applies styles when ...

✓ **the viewport is 768px wide or NARROWER**

- the viewport is exactly 768px
- the font is 768px
- the device is mobile-only

CSS Custom Properties (Variables)

WEEK 3 · FRIDAY

Define once in :root, use everywhere with var()

CSS

```
:root {
  --primary:    #0f766e;
  --accent:     #14b8a6;
  --bg:         #f0fdfa;
  --text:       #1e293b;
  --radius:     8px;
  --shadow:     0 4px 16px rgba(0,0,0,0.12);
}

.btn { background: var(--primary); border-radius: var(--radius); }
.card { background: var(--bg); box-shadow: var(--shadow); }
h1 { color: var(--primary); }

@media (prefers-color-scheme: dark) {
  :root { --bg: #051a0a; --text: #d4edda; }
}
```

Why variables?

One change updates everywhere

Change --primary once: every button, link, card updates instantly.

Easy dark mode

Redefine variables in @media: no class changes, no JS needed.

Component theming

Override per-component:
.card { --accent: red; }

Runtime JS access

```
el.style.setProperty('--primary','blue')
```

CSS Variables — Token Flow Diagram

WEEK 3 · FRIDAY

How :root variables cascade to every element

```
:root {  
  --primary: #0f766e;  --accent: #14b8a6;  --radius: 8px;  
}
```

These variables are available to EVERY element on the page (they cascade downward through the entire DOM tree)

.btn

```
background: var(--primary)  
border-radius: var(--radius)
```

.card

```
background: var(--bg)  
box-shadow: var(--shadow)
```

h1, h2

```
color: var(--primary)  
font-size: var(--heading)
```

**:root
(dark)**

```
--primary: #7dd3d0  
--bg: #0a1a1a
```

Dark Mode — just redefine the same variable names:

CSS

```
@media (prefers-color-scheme: dark) {  
  :root { --primary: #7dd3d0; --bg: #0a1a1a; --text: #e2f8f6; }  
} /* All .btn, .card, h1 — automatically update. Zero extra CSS needed. */
```

CSS Media Queries

WEEK 3 · FRIDAY

Apply styles only when certain conditions are met (responsive design)

```
@media ( condition ) { /* CSS rules that only apply when condition is true */ }
```

Mobile

```
@media (max-width: 480px)
```

≤ 480px

Tablet

```
@media (max-width: 768px)
```

≤ 768px

Desktop

```
@media (min-width: 1024px)
```

≥ 1024px

Wide

```
@media (min-width: 1440px)
```

≥ 1440px

CSS

```
/* Mobile-first approach */
.grid { display: block; }

@media (min-width: 768px) {
  .grid { display: flex; gap: 20px; }
}

@media (min-width: 1024px) {
  .grid { display: grid; grid-template-columns: repeat(3, 1fr); }
}
```

Other media features

- (orientation: landscape)
- (prefers-color-scheme: dark)
- (prefers-reduced-motion)
- print – styles for printing
- screen – default, any display

Responsive Design — Three Layouts

Same HTML, different CSS layout at each breakpoint

Mobile ≤ 480px

Card 1

Card 2

Card 3

Card 4

Tablet ≤ 768px

Card 1

Card 3

Card 2

Card 4

Desktop ≥ 1024px

C1

C2

C3

Full-width card

CSS

```
/* Mobile-first — base CSS targets mobile */
/* Mobile — 1 column, 1 row (flex row) */
/* Tablet — 2 columns, 2 rows (grid) */
/* Desktop — 3 columns, 2 rows (grid) */
```

Flexbox — Deeper Dive

WEEK 3 · FRIDAY

flex-direction · flex-wrap · order · flex-grow · gap

flex-direction

row (default) / column
row-reverse / column-reverse

flex-wrap

nowrap (default) / wrap
wrap-reverse

order

Default: 0 · lower = earlier
Visual reorder without changing HTML

flex-grow

0 = don't grow / 1 = fill space
flex: 1 = grow:1, shrink:1, basis:0

align-self

Override align-items for ONE item
auto · flex-start · center · flex-end

gap

Space between items (row & column)
Replaces margin hacks — cleaner

CSS

```
.container { display:flex; flex-wrap:wrap; gap:16px; }  
.item      { flex: 1; min-width: 200px; }  
.featured  { order: -1; flex: 2; }
```

CSS Grid — Introduction

WEEK 3 · FRIDAY

Two-dimensional layout: rows AND columns at the same time

Flexbox = 1 axis (row OR column) Grid = 2 axes (rows AND columns together)

CSS

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: auto auto;
  gap: 20px;
}

.hero { grid-column: 1 / 3; }
.sidebar { grid-row: 1 / 3; }

.layout {
  grid-template-areas:
    'header header header'
    'sidebar main main'
    'footer footer footer';
}

.main-content { grid-area: main; }
```

Key properties

grid-template-columns

Defines column widths
repeat(3,1fr) · 200px 1fr 200px

grid-template-rows

Defines row heights
auto · 100px · minmax(100px,auto)

gap

Space between cells
16px or 12px 20px (row col)

grid-column

Which columns item spans
1 / 3 or span 2

grid-row

Which rows item spans
1 / 4 or span 2

grid-template-areas

Named layout zones — most readable way to define a layout

CSS Grid — Named Areas Anatomy

WEEK 3 · FRIDAY

Visual breakdown of grid-template-areas in action

The CSS

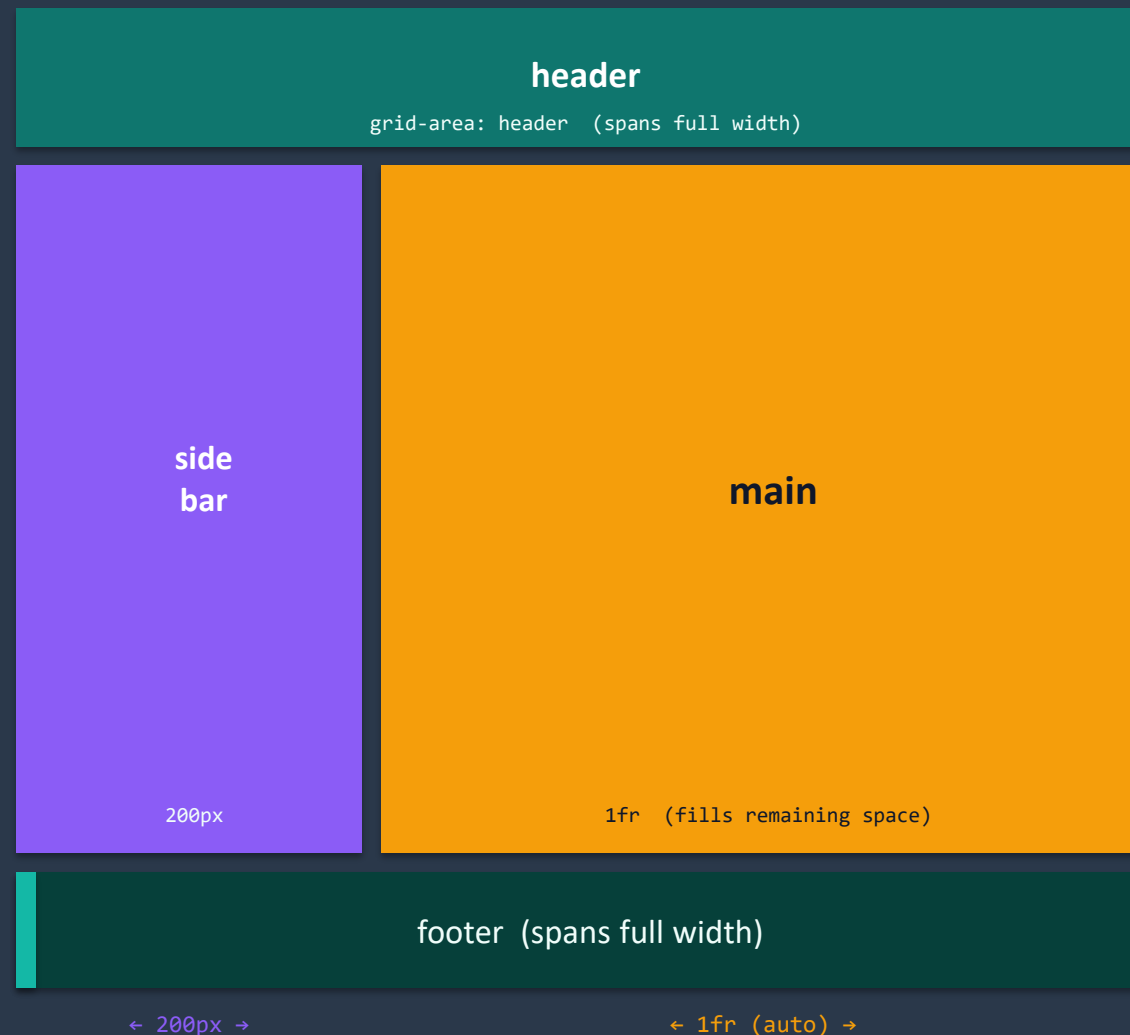
CSS

```
.layout {  
  display: grid;  
  grid-template-columns: 200px 1fr;  
  grid-template-rows:  
    60px auto 50px;  
  grid-template-areas:  
    'header header'  
    'sidebar main'  
    'footer footer';  
  gap: 12px;  
}
```

```
header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
main { grid-area: main; }  
footer { grid-area: footer; }
```



The Result



Check-In · Grid, Flexbox Advanced, border-radius, box-shadow

WEEK 3 · FRIDAY

Q18 flex-direction: column makes flex items ...

stack vertically (a column)
wrap to the next row
reverse their order
shrink to minimum width

Q19 flex-wrap: wrap allows flex items to ...

move to a new row when they overflow
shrink below minimum size
overlap each other
ignore the gap property

Q20 display: grid on a container ...

activates the 2-dimensional Grid layout
makes children flex items
hides overflowing children
stacks children absolutely

Q21 grid-template-columns: repeat(3, 1fr) creates ...

3 equal-width columns
1 column of 3fr width
3 rows
a fixed 300px grid

Q22 Which property adds space between grid OR flex items?

gap
margin
padding
spacing

Q23 border-radius: 50% on a square element creates ...

a perfect circle
a rounded rectangle
an oval
a hexagon

Q24 In box-shadow: 2px 4px 8px rgba(0,0,0,0.2) the 3rd value (8px) controls ...

blur radius
horizontal offset
vertical offset
spread radius

Q25 The CSS inherit keyword makes a property ...

take the computed value from the parent element
reset to browser default
apply !important override
copy from the next sibling

Check-In · Grid, Flexbox Advanced, border-radius, box-shadow

WEEK 3 · FRIDAY

Q18 flex-direction: column makes flex items ...

✓ **stack vertically (a column)**

wrap to the next row
reverse their order
shrink to minimum width

Q19 flex-wrap: wrap allows flex items to ...

✓ **move to a new row when they overflow**

shrink below minimum size
overlap each other
ignore the gap property

Q20 display: grid on a container ...

✓ **activates the 2-dimensional Grid layout**

makes children flex items
hides overflowing children
stacks children absolutely

Q21 grid-template-columns: repeat(3, 1fr) creates ...

✓ **3 equal-width columns**

1 column of 3fr width
3 rows
a fixed 300px grid

Q22 Which property adds space between grid OR flex items?

✓ **gap**

margin
padding
spacing

Q23 border-radius: 50% on a square element creates ...

✓ **a perfect circle**

a rounded rectangle
an oval
a hexagon

Q24 In box-shadow: 2px 4px 8px rgba(0,0,0,0.2) the 3rd value (8px) controls ...

✓ **blur radius**

horizontal offset
vertical offset
spread radius

Q25 The CSS inherit keyword makes a property ...

✓ **take the computed value from the parent element**

reset to browser default
apply !important override
copy from the next sibling

What you can now do:

Position

Use static/relative/absolute/fixed/sticky to place elements precisely

Pseudo

Style elements by state (:hover :focus) and insert content (::before ::after)

Transition

Animate property changes smoothly with transition: property duration easing

Animation

Create multi-step effects with @keyframes and the animation shorthand

Variables

Define --custom-properties in :root and use them everywhere with var()

Responsive

Use @media (max-width: ...) to apply styles only at certain viewport sizes

Grid

Create 2-dimensional layouts with display:grid and grid-template-columns